COMS 4170
Software Testing
Spring 2026

Chapter 3
Test Automation

1

Assignment One

- Opened discussion board on Canvas
- Discussion of Handin
  - Should submit only a .pdf file. Need to show screen shots that demonstrate you have completed the assignment
  - e.g. when ask to show report provide both the summary coverage and the code view.

2

Exercise One Discussion

- Branch and Line Coverage

JaCoCo Maven plug-in example with Offline Instrumentation > default

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType		67%		45%	10	13	4	10	1	2	0	1
Triangle		100%		n/a	0	1	0	2	0	1	0	1
Total	16 of 76	78%	12 of 22	45%	10	14	4	12	1	3	0	2

3

TriangleType.java

```
1.  //Introduction to Software Testing
2.  //Authors: Paul Ammann & Jeff Offutt
3.  //modified by M.Cohen for class exercise
4.  //Chapter 6;
5.  // See TriangleTypeTest for Junit tests
6.
7.
8.
9.
10. public class TriangleType
11. {
12.     /**
13.     * @param s1, s2, s3:  sides of the putative triangle
14.     * @return enum describing type of triangle
15.     */
16.   public static Triangle triangle (int s1, int s2, int s3)
17.   {
18.       // Reject non-positive sides
19.       if (s1 <= 0 || s2 <= 0 || s3 <= 0)
20.           return (Triangle.INVALID);
21.
22.       // Check triangle inequality
23.       if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
24.           return (Triangle.INVALID);
25.
26.       // Identify equilateral triangles
27.       if ((s1 == s2) && (s2 == s3))
28.           return Triangle.EQUILATERAL;
29.
30.       // Identify isosceles triangles
31.       // original program line is below (correct version)
32.       // if ((s1 == s2) || (s2 == s3) || (s1 == s3))
33.
34.       //fault introduced below by M.Cohen
35.       if ((s1 == s2) || (s2 == s3) || (s3 == s2))
36.           return Triangle.ISOSCELES;
37.
38.       return (Triangle.SCALENE);
39.   }
40. }
```

4

Covering Class Constructor

```
@Test public void createTriangle() {
TriangleType myTriangle = new TriangleType();
assertEquals(TriangleType.class, myTriangle.getClass());
}
```

```
8.
9.
10. public class TriangleType
11. {
12.     /**
13.     * @param s1, s2, s3:  sides of the putative triangle
14.     * @return enum describing type of triangle
15.     */
16.    public static Triangle triangle (int s1, int s2, int s3)
17.    {
18.       // Reject non-positive sides
19.       if (s1 <= 0 || s2 <= 0 || s3 <= 0)
20.          return (Triangle.INVALID);
21.
```

5

Questions from Exercise

Change the first program check (starting on line 18):

```
// Reject non-positive sides
   //correct code - comment out
    //     if (s1 <= 0 || s2 <= 0 || s3 <= 0)

// new fault    - last condition wrong
    if (s1 <= 0 || s2 <= 0 || s3 < 0)

     return (Triangle.INVALID);
```

Can you find the fault?

6

```
public class TriangleType
{
     /**
     * @param s1, s2, s3:  sides of the putative triangle
     * @return enum describing type of triangle
     */
    public static Triangle triangle (int s1, int s2, int s3)
    {
       // Reject non-positive sides
       if (s1 <= 0 || s2 <= 0 || s3 <= 0)
          return (Triangle.INVALID);

       // Check triangle inequality
       if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
          return (Triangle.INVALID);
```

Test (3,3,0, INVALID)

7

Running

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running TriangleTypeTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s - in TriangleTypeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

RIPR? What is happening?

8

Test Automation

- From our book: "*The use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test pre-conditions, and other test control and reporting functions*"
- Test generation: Another aspect of automation that can be achieved.
- We will see some automated test generation tools in this class

9

Benefits of Automation

- Reduces cost of testing
- Reduces time of testing – we can test more!
- Reduces human error
- Makes regression testing easier
  - can be executed by click of a button

10

Types of SE Tasks

- Revenue

  Contribute directly to the solution of a problem
- Excise

  Do not contribute directly to the solution

11

Examples

- Compiling a Java class

  Excise task – necessary but not helping the problem solution
- Determining methods that are needed to define a data structure

  Revenue task –necessary and directly helps to solve the problem

12

Automation

- Excise tasks are natural candidates for automation
- Software testing has more excise task than most parts of the software process
  - Maintaining test scripts
  - re-running tests
  - comparing expected results to oracle

All consume a lot of engineer time so if we can automate we benefit

13

Automation

What about test generation?

14

Automation

- Frees up engineer time so they can focus on the interesting work (revenue tasks)
- Allows tests to be run and re-run thousands of times and often (continuous integration)
- Helps eliminate errors of omission (e.g. failing to update new expected results)

15

Software Testability

- Estimates how likely testing will reveal a fault if one exists

  i.e. how *easy or hard a software system is* to test

16

Definitions

- Testability

The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine if these criteria have been met

Determined by two common practical issues

- Observability
- Controllability

17

Definitions

Software Observability

How easy it is to observe the behavior of a program in terms of its outputs, effects on the environment, and other hardware and software components.

18

Definitions

- Software Controllability

How easy it is to provide a program with the needed inputs in terms of values, operations and behaviors

19

Example

Embedded software

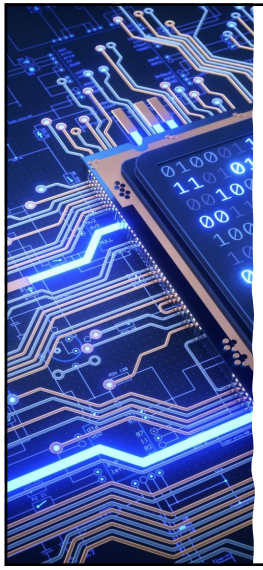
- Often does not provide human readable output, but impacts the hardware

  low observability

- Since it gets its inputs from hardware sensors (rather than a keyboard)

  low controllability

20



Simulation

- Used for many controllability and observability problems
- Can simulate a car crash and/or exceptional hardware values that might not be feasible with a working sensor

21

Example

22

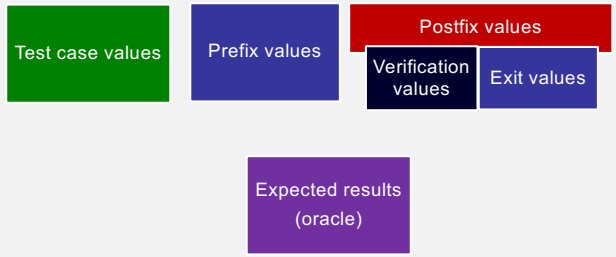
Our Triangle Program

- High controllability
- High observability

```
-------------------------------------------------------
 T E S T S
-------------------------------------------------------
Running TriangleTypeTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 sec

Results :

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.863 s
[INFO] Finished at: 2025-02-03T23:41:22-06:00
[INFO] ------------------------------------------------------------------------
```

23

Components of a Test Case



24

Components of a Test Case

Test case values

- The inputs used to complete an execution of the test
- These can be simple values, sequences, a web page, etc.

25

Help with Controllability and Observability

- Prefix Values
  - Inputs necessary to put the software into an appropriate state to receive the test case values (initialization)
- Postfix Values
  - Inputs that need to be sent after the test runs (finalization)

26

Example

mobile phone

- Prefix Values
  - Turning phone on
  - Opening dial pad
- Postfix Values
  - Pressing talk and end buttons

27

Two Types of Postfix Values

- Verification Values
  - Necessary to see the results of the test case
- Exit values
  - Values or commands needed to terminate the program or return it to a stable state

28

Test Oracle

Expected Results

- Decides whether or not a test has passed

29

Test Case Definition

Composed of the *test case values, prefix values, postfix values, and expected results* necessary for completing execution and evaluation of the software under test (sometimes called the SUT)

30

Test Set

A set of test cases called a test suite

31

Executable Test Script

A test case/set that is prepared in a form to be executed automatically on the test software and produce a report

32

RIPR Model Revisited

- Tests are designed to look for a fault in a particular location. The prefix values provide reachability (R), the test case values achieve infection (I), the postfix values achieve propagation (P) and the expected results reveal failures (R).

33

Example

estimateShipping()

estimates shipping charges for preferred customers in an automated shopping cart app

Goal:

Write tests to determine if estimated charges match actual charges

34

Example

- Test case values: (Infection)
  - Type of shipping (e.g. overnight)
- Prefix: (Reachability)
  - create shopping cart, add items to it, get preferred customer object and address
- Postfix: (Propagation)
  - Completing order so charges are computed
- Expected Values: (Revealability)
  - Extract actual shipping charge

35

Activities we can Automate

- Prepare test inputs as executable tests
  - can use shell scripts, batch files, input files or a tool that can control the software

Ultimately the output should be a report of passing/failing along with debug info

36

PART II

37

Test Automation Framework

- A set of assumptions, concepts and tools that support test automation
- Should include test driver (runs a test set by executing the software on each test). Should also compare results
- Simplest test driver- main() method of a class
  - Creates instances, manipulate values, etc.

38

Characteristics of Test Automation Frameworks

- Assertions to evaluate expected results
- Ability to share common test data between tests
- Test sets to easily organize and run tests
- Ability to run tests from either command line or GUI

39

The JUnit Test Framework

- JUnit test scripts
  - Run as standalone Java programs (command line or from within an IDE)
- Can test an entire class, a method, interacting methods, or interactions between objects (integration)
- Tests put into Test Methods
- Test methods put into a Test Class

40

Two Parts to JUnit

1. A collection of test methods
2. Methods to set up the program state (prefix values) and update after the test (postfix values)

41

JUnit

- Tests written from methods in the:
  - Junit.framework.assert class
- Each test checks a condition (assertion) and reports pass/fail to the test runner

42

Common Methods

assertTrue (Boolean)

simplest form (returns T/F)

assertTrue (String, Boolean)

provides information to tester if the assertion is false (failure)

fail(String)

used specify a failing message when a specific line is reached

43

Example of fail()

```
@Test
public void customInterceptorAppliesWithCheckedException() {
  try {
    this.cs.throwChecked(0L);
    fail("Should have failed");
  }
  catch (RuntimeException e) {
    assertNotNull("missing original exception", e.getCause());
    assertEquals(IOException.class, e.getCause().getClass());
  }
  catch (Exception e) {
    fail("Wrong exception type " + e);
  }
}
```

44

Test Fixture

- State of the test defined by the current values of key variables in the software
- Can be shared between tests
- Objects used in fixtures are declared as instance variables and initialized in a @Before method and reset or deallocated in an @After method

45

Example

```
Public class Calc{

  static public int add (int a, int b)
  {
    return a+b
  }
}

'---------------------------------------'

import org.junit*;
import static org.junit.Assert.*;
public class CalcTest
{
    @Test public void testAdd()
    {
        assertEquals("Calc sum incorrect", 5==Calc.add(2,3));
    }
}
```

Expected output

Test values

Printed if assert fails

46

```
public static <T extends Comparable<? super T>> T min (List<? extends T> list)
    {
      if (list.size() == 0)
      {
        throw new IllegalArgumentException ("Min.min");
      }
      Iterator<? extends T> itr = list.iterator();
      T result = itr.next();

      if (result == null) throw new NullPointerException ("Min.min");

      while (itr.hasNext())
      {  // throws NPE, CCE as needed
        T comp = itr.next();
        if (comp.compareTo (result) < 0)
        {
          result = comp;
      } }
      return result;
}
```


47

MinTest Class

- Standard imports for all JUnit classes :

```
import static org.junit.Assert.*;
import org.junit.*;
import java.util.*;
```

- Test fixture and pre-test setup method (prefix) :

```
private List<String> list;   // Test fixture

// Set up - Called before every test method.
@Before
 public void setUp()
 {
    list = new ArrayList<String>();
 }
```

- Post test teardown method (postfix) :

```
// Tear down - Called after every test method.
@After
public void tearDown()
{
   list = null;   // redundant in this example
}
```


48

Min Test Cases: NullPointerException

```
@Test public void testForNullList()
{
  list = null;
  try {
     Min.min (list);
  } catch (NullPointerException e) {
     return;
  }
  fail ("NullPointerException expecte
}
```

This NullPointerException test decorates the @Test annotation with the class of the exception

```
@Test (expected = NullPointerException.class)
public void testForNullElement()
{
   list.add (null);
   list.add ("cat");
   Min.min (list);
}
```

This NullPointerException test uses the fail assertion

```
@Test (expected = NullPointerException.class)
public void testForSoloNullElement()
{
   list.add (null);
   Min.min (list);
}
```

This NullPointerException test catches an easily overlooked special case

49

More Exception Test Cases for Min

```
@Test (expected = ClassCastException.class)
@SuppressWarnings ("unchecked")
public void testMutuallyIncomparable()
{
  List list = new ArrayList();
  list.add ("cat");
  list.add ("dog");
  list.add (1);
  Min.min (list);
}
```

Note that Java generics don't prevent clients from using raw types!

```
@Test (expected = IllegalArgumentException.class)
public void testEmptyList()
{
   Min.min (list);
}
```

Special case: Testing for the empty list

50

s for Min

```
@Test
public void testSingleElement()
{
   list.add ("cat");
   Object obj = Min.min (list);
   assertTrue ("Single Element List", obj.equals ("cat"));
}

@Test
public void testDoubleElement()
{
   list.add ("dog");
   list.add ("cat");
   Object obj = Min.min (list);
   assertTrue ("Double Element List", obj.equals ("cat"));
}
```

Finally! A couple of "Happy Path" tests

51

Summary: Seven Tests for Min

- Five tests with exceptions
  1. null list
  2. null element with multiple elements
  3. null single element
  4. incomparable types
  5. empty elements
- Two without exceptions
  6. single element
  7. two elements

52

Data Driven Tests

- Same test needs to run with different inputs (e.g. our triangle program)
- Write tests once and supply data in a table
- Called data-driven testing

53

```
import org.junit.*;
import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;
import org.junit.runners.Parameterized.Parameters;
import static org.junit.Assert.*;
import java.util.*;

@RunWith (Parameterized.class)
public class DataDrivenCalcTest
{  public int a, b, sum;

   public DataDrivenCalcTest (int v1, int v2, int expected)
   { this.a = v1; this.b = v2; this.sum = expected; }

   @Parameters public static Collection<Object[]> parameters()
   { return Arrays.asList (new Object [][] {{1, 1, 2}, {2, 3, 5}}); }

   @Test public void additionTest()
   { assertTrue ("Addition Test", sum == Calc.add (a, b)); }
}
```

Constructor is called for each triple of values

Test 1
Test values: 1, 1
Expected: 2

Test 2
Test values: 2, 3
Expected: 5

Test method

54

Running JUnit on Command Line

- This is all we need to run JUnit in an IDE (like Eclipse)
- We need a main() for command line execution …

55

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import junit.framework.JUnit4TestAdapter;

// This section declares all of the test classes in the program.
@RunWith (Suite.class)
@Suite.SuiteClasses ({ StackTest.class })  // Add test classes here.

public class AllTests
{
   // Execution begins in main(). This test class executes a
   // test runner that tells the tester if any fail.
   public static void main (String[] args)
   {
      junit.textui.TestRunner.run (suite());
   }

   // The suite() method helps when using JUnit 3 Test Runners or Ant.
   public static junit.framework.Test suite()
   {
      return new JUnit4TestAdapter (AllTests.class);
   }
}
```


56

Tests with Parameters: JUnit Theories

Suppose you want to test

$$\forall x \in X \cdot P(x) \rightarrow Q(x)$$

Means: for all values in a particular domain, X, if the precondition P holds, then the postcondition Q also holds

```
@Theory public void removeThenAddDoesNotChangeSet (
        Set<String> someSet, String str) {          // Parameters!
    assumeTrue (someSet != null)                    // Assume
    assumeTrue (someSet.contains (str)) ;           // Assume
    Set<String> copy = new HashSet<String>(someSet); // Act
    copy.remove (str);
    copy.add (str);
    assertTrue (someSet.equals (copy));             // Assert
}
```


Question: Where Do The Data Values Come From?

- Answer:
  - All combinations of values from @DataPoints annotations where assume clause is true

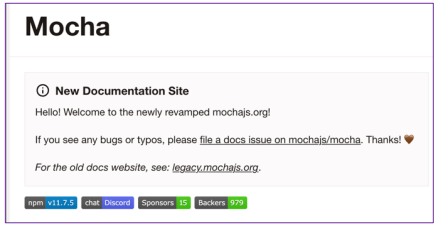
```
@DataPoints
public static String[] animals = {"ant", "bat", "cat"};

@DataPoints
public static Set[] animalSets = {
  new HashSet (Arrays.asList ("ant", "bat")),
  new HashSet (Arrays.asList ("bat", "cat", "dog", "elk")),
  new HashSet (Arrays.asList ("Snap", "Crackle", "Pop"))
};
```

Nine combinations of animalSets[i].contains (animals[j]) is false for five combinations

Summary

- The only way to make testing efficient as well as effective is to automate as much as possible
- Test frameworks provide very simple ways to automate our tests
- It is no "silver bullet" however … it does not solve the hard problem of testing:
- This is test design … the purpose of test criteria

Other (example) Unit Test Frameworks



Example PyTest

```
import time
from enum import Enum

class TriangleType(Enum):
    INVALID, EQUILATERAL, ISOSCELES, SCALENE = 0, 1, 2, 3

def classify_triangle(a, b, c):
    # Sort the sides so that a <= b <= c
    if a > b:
        tmp = a
        a = b
        b = tmp

    if a > c:
        tmp = b
        a = c
        c = tmp

    if b > c:
        tmp = b
        b = c
        c = tmp

    if a + b <= c:
        return TriangleType.INVALID
    elif a == b and b == c:
        return TriangleType.EQUILATERAL
    elif a == b or b == c:
        return TriangleType.ISOSCELES
    else:
        return TriangleType.SCALENE

if __name__ == "__main__": # pragma: no cover
    import sys
    classify_triangle(int(sys.argv[1]), int(sys.argv[2]), int(sys.argv[3]))
```

61

Example PyTest

```
import time
import pytest
from triangle import TriangleType, classify_triangle


def check_classification(triangles, expected_result):
    for triangle in triangles:
        assert classify_triangle(*triangle) == expected_result


def test_invalid_triangles():
    triangles = [(1, 2, 9), (1, 9, 2), (2, 1, 9), (2, 9, 1), (9, 1, 2),
                 (9, 2, 1), (1, 1, -1), (1, -1, 1), (-1, 1, 1)]
    check_classification(triangles, TriangleType.INVALID)


def test_equilateral_triangles():
    triangles = [(1, 1, 1), (100, 100, 100), (99, 99, 99)]
    check_classification(triangles, TriangleType.EQUILATERAL)


def test_isosceles_triangles():
    triangles = [(100, 99, 99), (99, 100, 99), (99, 99, 100), (3, 3, 2), (3, 2, 3), (2, 3, 3)]

    check_classification(triangles, TriangleType.ISOCELES)


def test_scalene_triangles():
    triangles = [(5, 4, 3), (5, 3, 4), (4, 5, 3), (4, 3, 5), (3, 5, 4)]
    check_classification(triangles, TriangleType.SCALENE)


@pytest.fixture(scope="session", autouse=True)
def starter(request):
    start_time = time.time()

    def finalizer():
        print("runtime: {}".format(str(time.time() - start_time)))

    request.addfinalizer(finalizer)
```

62

Results

```
mcohen> pytest -s test_triangle.py
```

```
test_triangle.py:14:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

triangles = [(1, 2, 9), (1, 9, 2), (2, 1, 9), (2, 9, 1), (9, 1, 2), (9, 2, 1), ...]
expected_result = <TriangleType.INVALID: 0>

    def check_classification(triangles, expected_result):
        for triangle in triangles:
>           assert classify_triangle(*triangle) == expected_result
E           assert <TriangleType.ISOSCELES: 2> == <TriangleType.INVALID: 0>
E            +  where <TriangleType.ISOSCELES: 2> = classify_triangle(*(2, 9, 1))

test_triangle.py:8: AssertionError
_____________________________ test_equalateral_triangles ______________________________

    def test_equalateral_triangles():
        triangles = [(1, 1, 1), (100, 100, 100), (99, 99, 99)]
>       check_classification(triangles, TriangleType.EQUALATERAL)
E       AttributeError: type object 'TriangleType' has no attribute 'EQUALATERAL'. Did
you mean: 'EQUILATERAL'?
```

63